

Axiom Governance for an AI-Generated Formal Library: Taxonomy, Redemption, Three-Caliber Reconciliation, and Honest Proof-Rate Metrics*

TOE-SYLVA Formalization Governance Group[†]

August 21, 2026 (D4 full draft; canonical numbers per P0 reconciliation)

Abstract

AI-assisted formalization programs can scale proof-term production faster than they scale proof *verification*. We report the governance methodology of TOE-SYLVA, a Lean 4 corpus built on pinned mathlib4 that accumulated, alongside genuine developments, a large ledger of `axiom` declarations used as deferred-proof placeholders, plus a machine-generated tier of 119,859 modules. The methodology has four components. (1) *Axiom audit*: a machine-readable registry of every non-batch `axiom` (448 entries at baseline) with a four-class taxonomy separating genuine mathematical assumptions from definitional shorthand and vacuous placeholders. (2) *Redemption sweeps with semantic re-grading*: six sweeps closed 196 entries (30+44+31+29+30+32); the P0 reconciliation re-grades them into five classes — 50 vacuous placeholder closures, 90 $P \rightarrow P$ conditionalizations, 17 data-to-definition conversions, 24 definitional reconstructions, and 13 substantive proofs (plus 2 bundling restructurings excluded from the content count) — an intentionally unflattering decomposition in which 71.4% of closures transfer no proof content. (3) *Three-caliber reconciliation*: registry (448), live-declaration (253), and sweep-process (196) accounts are reconciled to exact identities, and six documented caliber conflicts (axiom counts, Agda postulates, dual proof-rate dashboards disagreeing by 25×, a 30.2% “sorry contamination” alarm that reversed to zero under a code-level caliber, an internal ledger miscount, and double-counted closures) are each resolved with named calibers. (4) *An honesty layer*: claim tiers with falsifiability criteria, an honest proof-rate metric (3.91% = 22,328/571,611 on the handcrafted corpus; batch tier reported separately at 0%; a legacy 99.79% dashboard is preserved, marked stale), a quarantine protocol, and a CI gate that fails on silent deletions. Two case studies — a CLAIM→THEOREM upgrade along a Chern–Simons chain with explicit mathlib-delegation labels, and a Dedekind-domain segment honestly registered as zero-new-mathematics delegation — exhibit the full lifecycle. The transferable claim: in mixed human/AI proof libraries, metrics are trustworthy only as (number, caliber) pairs, with calibers declared, machine-checkable, and CI-enforced.

*Target venue: Journal of Automated Reasoning (main). Draft status: full first submission draft (D4), 2026-08-21. The library and this manuscript were produced with AI assistance under human tasking and review; the transparency conventions are described in Sec. 1.5.

[†]Correspondence: repository maintainers. Artifact chain: `framework/paper/` (this manuscript + build log), `framework/p0_reconciliation_report.md` (canonical numbers), `scripts/verify_honest_repo.py` (honesty CI).

Draft status (D4: full first submission draft). This manuscript expands the D3 compilation skeleton into a complete draft targeted at JAR (per the D2 journal-matching matrix), including the two P1 gap items from the D1 internal review: a related-work chapter (Sec. 2) and an expanded limitations section with honest disclosure of the $P \rightarrow P$ and vacuous classes (Sec. 9.2). All headline figures are *canonical* per the P0 reconciliation report (`framework/p0_reconciliation_report.md`, 2026-08-21): sweep accounting $196 = 30+44+31+29+30+32$; registry 448; residual/live 253; five classes 50/90/17/24/13 with 2 bundling entries excluded; Agda 131. Values in red angle brackets $\langle \dots \rangle$ are declared v7.98/v7.99 working-tree baselines outside the P0 register’s scope, retained for traceability; the repository source of each is annotated in the L^AT_EX source comments. The conflict inventory (C1–C6) that motivated the caliber discipline is tabulated in Sec. 5.2.

Contents

1	Introduction	4
1.1	Setting: a library that scaled faster than its governance	4
1.2	The governance program	4
1.3	Contributions	5
1.4	Paper roadmap	6
1.5	Transparency statement	6
2	Related Work	6
2.1	Large-scale formal libraries and their quality culture	6
2.2	LLM-generated proofs and autoformalization integrity	7
2.3	Audit precedents inside the repository	7
2.4	What is new here	7
3	The Axiom Registry Audit	8
3.1	Corpus anatomy	8
3.2	The four-class taxonomy	8
3.3	What the registry buys	9
4	Redemption Sweeps and the Five-Class Taxonomy	9
4.1	The sweep protocol	9
4.2	The five semantic classes	10
4.3	The honesty of $P \rightarrow P$	11
4.4	Closing the declaration account	12
5	Three-Caliber Reconciliation	12
5.1	Why numbers drift	12
5.2	Axioms: three calibers that must all close	12
5.3	Agda: three calibers for one track	14
5.4	Proof rate: the 25× disagreement	14
5.5	The reconciliation as a first-class artifact	14
6	Case Study I: The Chern–Simons Chain and Theorem T3	15
6.1	The claim and the chain	15
6.2	The upgrade: what was proved, and how it is labeled	16
6.3	What T3 is not: gaps and the rollback incident	16
7	Case Study II: The Dedekind Delegation Layer	17
7.1	The course chain and the delegation pattern	17
7.2	Zero new mathematics, and why it is registered anyway	18
7.3	T3 vs. Dedekind: two ends of one scale	19

8	Honesty Governance	19
8.1	Claim tiers and falsifiability	19
8.2	The honest proof-rate metric	19
8.3	The sorry incident: a 30.2% contamination that wasn't	20
8.4	Quarantine, not deletion	20
8.5	The honesty CI and its explicit boundary	20
9	Discussion and Limitations	21
9.1	Caliber discipline as the central thesis	21
9.2	Limitations	21
9.3	Anticipated objections	22
9.4	Future work	23
9.5	Ethics and transparency	23
10	Conclusion	23
A	Reproduction Protocol	26
A.1	Axiom calibers	26
A.2	Agda calibers	26
A.3	Proof-rate calibers	27
B	Glossary of Calibers	27

1 Introduction

1.1 Setting: a library that scaled faster than its governance

TOE-SYLVA is a formalization program whose Lean 4 side is built strictly on top of a pinned `mathlib4` revision (commit `8a178386` at the v7.99 baseline; toolchain Lean 4.29.0) and contains two very different kinds of content. The first is a set of handcrafted developments targeting physics-adjacent mathematics (Chern–Simons theory, topological invariants, gauge theory, p -adic analysis, a course chain in algebraic number theory), grown to roughly `<571611>` declared theorems. The second is a machine-generated *batch corpus* of `<119859>` template modules produced by a generator whose manifest lists `<119831>` entries with SHA-256 sampling (`<500>` draws, zero mismatches at the v7.98 baseline). A third, smaller Agda track holds 7 files with 131 name-level `postulate` declarations (canonical, P0). The library, its audits, and this paper were all produced with AI assistance under human tasking and review; that fact is not a disclaimer but the paper’s subject matter. The defining event of the program is that the second kind of content — and, earlier, large parts of the first — scaled two to three orders of magnitude faster than the processes that were supposed to keep the corpus honest.

The result was not one problem but three, each a distinct failure mode of *measurement*, not of taste:

- F1 Axioms as invisible deferred proof.** An `axiom` command in Lean is a legal, type-checked, fully trusted placeholder. In a corpus where AI-assisted drafting produced plausible-looking statement blocks at scale, the non-batch corpus accumulated 448 registered `axiom` declarations (canonical registry caliber, 116 non-batch files), while top-level summaries continued to say “formalized”. “Formalized” and “proved” are different predicates; the library had been quietly conflating them.
- F2 Proof-rate metrics that drift with the measuring rule.** Over the program’s history the reported “proof rate” of the handcrafted corpus took the values `<88.08%>`, `<99.5%>`, `<99.9%>`, `<100%>`, `<99.79%>`, and 3.91% — a $25\times$ spread — with no change to the underlying proofs. Every value was “correct” under some counting rule; no rule was stated.
- F3 String-level scans that mistake negations for placeholders.** A repository-wide text search for `sorry` reported `<30.2%>` of the batch corpus as “containing sorry” (`<36199>` of `<119859>` files). A full-population code-level audit found the code-level count to be zero: 36,198 of the 36,199 hits were the fixed header comment *asserting the absence* of placeholders, and the single remaining hit was a theorem whose *name* contains the word. The scan was measuring comments, not proofs.

None of these failures is exotic. Each is what happens when a metric is quoted without the rule that produced it — a *caliber*, in the terminology this paper develops — and each would have survived casual review indefinitely, because the surface numbers were plausible. They were caught only because the program eventually built governance whose explicit job was to catch them.

1.2 The governance program

The response, executed over versions v7.94–v7.99 of the repository, is the subject of this paper. It has four stages, which are also the paper’s methodological mainline:

1. **Registry audit** (Sec. 3): every `axiom` declaration in the non-batch corpus is registered in a machine-readable JSON registry, classified into a four-class taxonomy (primitive / definitional / schema / placeholder), with per-entry provenance. The registry is the single authority for *what the library owes*: 448 entries at baseline.

2. **Redemption sweeps** (Sec. 4): six organized passes (sweep1–sweep6) convert registered axioms into theorems, definitions, or explicitly conditional statements, closing 196 entries in total ($30+44+31+29+30+32$). Every closure is re-graded *post hoc* by semantic content into a five-class taxonomy — vacuous placeholder closure (50), $P \rightarrow P$ conditionalization (90), data-to-definition conversion (17), definitional reconstruction by `rfl` (24), substantive proof (13), plus 2 axiom-bundling restructurings excluded from the redemption count — because the honest headline of the program is *what kind* of work those 196 closures actually were.
3. **Three-caliber reconciliation** (Sec. 5): the program’s own numbers are then audited against each other. The registry caliber (448), the live `.lean`-declaration caliber (253 residual), and the sweep-accounting caliber (196 cumulative) close to the identity $445 - 166 - 26 = 253$ at the declaration level, and $448 - 253 = 195 = 142 + 42 + 11$ at the registry level. Six documented caliber conflicts (C1–C6) are resolved and their anatomy tabulated.
4. **Honesty governance** (Secs. 6–8): a claim-tier system (THEOREM / THEOREM* / CLAIM / CONJECTURE) with falsifiability criteria, an honest proof-rate metric defined as a (number, caliber) pair, a machine-checked honesty CI, a quarantine protocol for the one genuinely contaminated module, and a deleted-claims ledger enforcing the no-silent-deletion rule.

Two lifecycle case studies exercise the whole stack end to end — the Chern–Simons chain, where a CLAIM→THEOREM upgrade is performed along a six-level dependency chain with explicit mathlib-delegation labels and an explicit statement of what the upgraded theorem is *not* (Sec. 6); and the Dedekind delegation, where a “theorem” that is a pure wrapper over a mathlib instance is registered honestly as such (Sec. 7).

1.3 Contributions

- C1** A four-class axiom taxonomy (primitive / definitional / schema / placeholder) with a machine-readable registry of 448 axioms over 116 non-batch files — the audit artifact that made F1 visible at all, and the precondition for every later number in the paper.
- C2** A redemption methodology — six sweeps closing 196 registered axioms — together with a five-class semantic re-classification (50 vacuous / 90 $P \rightarrow P$ / 17 data-def / 24 definitional / 13 substantive / 2 bundling excluded) that refuses the aggregate “196 redeemed” reading. The transferable claim is the *decomposition*, not the count.
- C3** A three-caliber reconciliation protocol that closes the registry (448), live-declaration (253), and sweep (196) accounts to a machine-reproducible identity, resolving six documented conflicts (C1–C6) — including a $25\times$ proof-rate disagreement and a $2.7\times$ Agda-postulate disagreement — without editing a single proof.
- C4** An honesty-governance layer: claim tiers with falsifiability criteria; an honest proof-rate metric whose reference implementation counts code-level placeholders only (`count_sorry_no_comments`); a repository CI that enforces the honesty layer’s invariants; and the full-population reversal of the 30.2% “sorry contamination” signal (F3), which we argue is a *positive* result: the caliber discipline caught a would-be crisis that string-level tooling had manufactured.
- C5** Two full lifecycle case studies — the Chern–Simons T3 upgrade with mathlib-delegation labels, gap accounting, and a concurrent-work rollback that the audit layer itself detected; and the Dedekind delegation, an honest registration of zero-new-math “theorems”. Together they show the tier labels doing real communicative work.

1.4 Paper roadmap

Section 2 positions the work against large-scale formalization practice (mathlib, Isabelle/AFP, Metamath) and the 2024–2026 literature on LLM-generated proofs and autoformalization integrity. Sections 3–5 are the methodological core: registry audit, redemption sweeps and the five-class taxonomy, and the three-caliber reconciliation. Sections 6 and 7 are the case studies. Section 8 assembles the honesty layer. Section 9 states limitations — including an explicit disclosure that 71.4% of closures (the vacuous and $P \rightarrow P$ classes) transfer no proof content — anticipated objections, and future work. Appendix A gives the reproduction protocol; Appendix B is a glossary of calibers.

1.5 Transparency statement

This library was produced with AI assistance, and so was this paper. The repository discipline it describes — registration first, boundaries explicit, no silent deletions — is applied to the manuscript itself. All headline figures (196, 448, 253, the five-class decomposition, Agda 131) are canonical per the P0 reconciliation report (`framework/p0_reconciliation_report.md`, 2026-08-21 [24]), and the remaining values in red angle brackets are declared v7.98/v7.99 working-tree baselines, each annotated with its repository source in the L^AT_EX source comments. Every claim about the library cites the internal audit report it rests on. The supplementary build artifacts (compile log, PDF) accompany the manuscript under `framework/paper/`.

2 Related Work

2.1 Large-scale formal libraries and their quality culture

The problems this paper addresses — placeholder declarations, metric drift, untracked deferred proof — are not specific to AI-assisted corpora, but AI assistance changes their *rate* and *scale*. The established communities have long maintained disciplines that our program independently rediscovers and then has to formalize more aggressively, because at machine-generation scale no informal convention survives.

mathlib/Lean 4 [15, 14] is the foundation layer of our corpus and the reference point for axiom discipline. The community norm is that a proof-development’s axiom footprint is inspectable with `#print axioms`, and that a result relying on nothing beyond Lean’s standard axioms (`propext`, `Classical.choice`, `Quot.sound`) is reported as such; that “standard trio” is exactly the certificate our case-study registry requires (Sec. 6). **mathlib**’s own growth is curated through review; our corpus deliberately contains an *unreviewed* tier, which is why we need a registry where a mature library needs code review. Our claim-tier THEOREM* — mathematics proved elsewhere, formalized elsewhere, and only *delegated to* by our corpus — is modeled on the ordinary situation of a downstream development citing **mathlib** results; the repository’s FLT registration (formal Fermat’s Last Theorem as delegated by **mathlib**, roughly (12000) lines of **mathlib** proof atop a large dependency cone — the figure is the repository’s registration, not an independent measurement) is the canonical example.

Isabelle/HOL and the Archive of Formal Proofs [13] pair a mature proof assistant with a refereed, versioned archive: entries enter AFP through review and are then kept building against the evolving Isabelle release — the closest existing analogue to our “registration before promotion” rule, but with human referees where we use machine audit plus tier labels. The AFP’s review gate exists precisely because unrestricted growth degrades trust; our batch corpus is the limiting case of unrestricted growth, and its quarantine protocol (Sec. 8.4) is a machine-scale substitute for that gate.

Metamath [8] represents the strictest tradition of axiom economy: every proof step is an explicit substitution instance of a small, fixed axiom set, so the notion of “untracked trust”

cannot arise by construction. Our registry taxonomy (Sec. 3.2) can be read as retrofitting a weaker version of that strictness onto a corpus that was *not* built that way — a retrofit that only became necessary once AI-assisted drafting let statement volume outrun proof volume.

2.2 LLM-generated proofs and autoformalization integrity

A 2024–2026 literature line — twelve papers whose identifiers were verified in the repository’s 2026-08-06 scan [4, 10, 5, 6, 1, 28, 7, 11, 2, 3, 9, 12] — frames the exact failure surfaces this paper governs:

- **Assistance and generation.** Lean Copilot [4] and multi-agent formalization frameworks such as MASA [7] put LLMs inside the proof-writing loop; corpora built this way inherit the LLM’s statement-level fluency without any guarantee of proof-level content — the precondition of failure mode F1. Process-driven benchmarks (FormL4 [10]) and corpus-build methodologies (Lean Workbook [5], common-framework proposals [28]) study how NL→Lean pipelines scale.
- **Evaluation and its blind spots.** Research-grade evaluation suites for real Lean projects (RLMEval [11]), in-the-wild autoformalization studies [1], intent-capture evaluation [6], and robustness studies showing that frontier models fail under mere paraphrase [9] or on non-canonical real proof texts [12] all converge on one point: *curated-benchmark performance does not transfer to unguarded corpora*. Our batch corpus is precisely an unguarded corpus, and Sec. 8.3 shows what a naive scan of it reports.
- **Semantic hallucination.** The most directly relevant result for us: type-checking success does not imply that a formalized proposition faithfully expresses the intended mathematics — “semantic hallucination” in the terminology of the Lean Atlas study [3], with formal verification of LLM-generated outputs proposed as the mitigation [2]. Our claim tiers and falsifiability criteria (Sec. 8.1) are a governance-scale answer to the same problem: the registry records *what* a statement claims and at what tier, so that a type-checked placeholder cannot masquerade as a theorem.

To our knowledge, the autoformalization literature documents the *capability* side of this problem extensively, but not the *corpus-governance* side: what to do once a mixed human/AI library already contains four hundred trusted placeholders and a hundred thousand generated modules. This paper is a case study of the governance side.

2.3 Audit precedents inside the repository

The program did not start from zero: the repository’s own history supplied the audit precedent, and the paper’s methodology is a generalization of it. An earlier counting-geometry audit identified and deleted 15 fabricated claims, logging them in a machine-readable deleted-claims ledger (`data/deleted_fabricated_claims.json`) together with the protocol document that distilled the failure-mode taxonomy of AI-generated mathematical writing. Four subsequent rounds of quality engineering verified (1943) arXiv identifiers, zeroed (25) failing verification scripts by fixing the underlying content rather than loosening the assertions, and repaired some (500) reference-text consistency defects (DEVELOPMENT_DIRECTIONS.md, repository history). The registry, sweeps, and reconciliation machinery of Secs. 3–5 is that practice scaled from “audit the papers” to “audit the library”; the deleted-claims ledger survives as a CI-enforced invariant (Sec. 8.5).

2.4 What is new here

Table 1 summarizes the delta against each conversation partner.

Table 1: Position relative to prior practice.

Partner	What this paper adds
mathlib/Lean discipline	A registry that makes the axiom footprint of an <i>unreviewed</i> , AI-assisted corpus explicit and auditable, plus a semantic re-grading of every “redemption” so that closure counts cannot launder placeholders into proofs.
AFP review culture	A machine-checkable substitute (claim tiers + honesty CI + quarantine) for review gates when the corpus grows faster than review capacity.
Metamath axiom economy	A post-hoc retrofit: taxonomy, per-entry provenance, and closing identities for a corpus that was not born under strict accounting.
Autoformalization integrity literature	The missing corpus-governance case study: a full reconciliation of registry, declaration, and process accounts in a mixed human/AI library, with the (number, caliber) discipline as the proposed interface.
Internal audit precedent	Generalization from document audits to library audits; the same ledger conventions, enforced by CI instead of by convention.

3 The Axiom Registry Audit

3.1 Corpus anatomy

The audited object is the *non-batch* Lean corpus: all `.lean` files under `sylnva_formalization/` except the generated `SYLVA_Proven*` batch modules (batch corpus: [\(119859\)](#) files, governed separately, Sec. 8.4). The non-batch corpus is handcrafted in the sense of “curated by humans with AI assistance”, covering physics-adjacent mathematics: Chern–Simons theory and the chern number family, gauge theory and instanton constructions, quantum Hall effect modules, cosmology (FLRW, dark energy, inflation), string/M-theory brane content, computability (Cook–Levin, CNF encodings, polynomial-time closures), p -adic analysis, a course chain in algebraic number theory, and collective-intelligence and learning-theory statements. Alongside it, the Agda track holds 7 files (Quantum, Spectrum, Complete, CNFCategory, Order, Cauchy, Field modules) with 131 name-level `postulate` declarations — canonical caliber; the per-module split is Quantum 53, Spectrum 36, Complete 15, CNFCategory 12, Order 12, Cauchy 2, Field 1.

The asymmetry that motivates a registry: in this corpus, an `axiom` command is not a mathematical foundation choice but a *deferred obligation* — typically an AI-drafted statement block that reads like a theorem, type-checks like a theorem, and imports the full trust of the ambient logic the moment anyone builds on it. Nothing in the Lean toolchain distinguishes “foundational axiom we choose to assume” from “proof we hope to write later”. The registry exists to restore that distinction.

3.2 The four-class taxonomy

Every `axiom` declaration in the non-batch corpus is registered in a machine-readable JSON registry (`framework/axiom_registry.json`, v2.0 [\[17\]](#)) with per-entry fields: file path, declaration name, statement, a taxonomy category, and (where applicable) a suggested reduction method. The taxonomy has four classes, defined by *why the axiom exists* rather than by subject matter:

- **Primitive** (250 entries): statements intended as genuine mathematical content — conjectures, physical input, or results whose proofs are currently out of reach (e.g. the spectral action or Chern–Weil statements). These are the corpus’s real debts.

- **Definitional** (142 entries): axioms that restate definitions — the “proof” is a definitional unfolding that nobody performed (e.g. a physical constant asserted as an axiom when it is the *definition* of that constant).
- **Placeholder** (42 entries): statements whose conclusion is `True` (or an implication into `True`) — pure syntactic placeholders, carrying no mathematical content at all.
- **Schema** (14 entries): cross-cutting structural assumptions that license a whole family of statements at once (e.g. the causal-network→gauge-level identification of the Chern–Simons chain, Sec. 6); tracked separately because redeeming them is a research program, not a proof task.

The registry carries 448 entries over 116 non-batch files, with 391 unique names and (57) cross-file duplicate registrations (canonical total; the decomposition 250/142/42/14 and the residual identities of Sec. 5 are canonical per the P0 report). Version 2.0 of the registry also carries an erratum trail in its metadata — e.g. an early note asserting “no `.agda` files exist” was corrected by the P0 reconciliation, which measured the Agda track directly (7 files, 131 postulates) — because the registry is itself subject to the audit discipline it enforces.

3.3 What the registry buys

Three things, each exploited in later sections:

1. **A census that cannot lie by aggregation.** The four classes are disjoint, machine-tallied, and reproducible from the registry JSON; “how much does the library owe?” acquires a defensible answer (448 at baseline: 250 primitive, 142 definitional, 42 placeholder, 14 schema) instead of the previous unbounded “350+” (an early headline later shown to be a caliber artifact, conflict C1, Sec. 5.2).
2. **A work queue with honest exit criteria.** Placeholder axioms can be discharged by trivial proofs *without changing any semantics*; definitional axioms can be discharged by refolding them as `defs`; primitives cannot be “swept” at all and must either be proved, conditioned, or left standing with their debt visible. This grading is what made the sweep program (Sec. 4) something other than number farming.
3. **An anchor for reconciliation.** Registry, live declarations, and process logs are three independent records of the same corpus; closing them against each other (Sec. 5) is only possible because the registry pins one of the three to a machine-readable ground truth.

The registry does *not* judge mathematics. It records obligations. The distinction between a deep primitive and a vacuous placeholder is a statement about *bookkeeping*, not about importance; the physics-facing primitives that remain unredeemed (239 at the time of writing, plus 14 schema — the canonical residual 253 of Sec. 5) are the library’s honest, visible debt.

4 Redemption Sweeps and the Five-Class Taxonomy

4.1 The sweep protocol

A *redemption sweep* is an organized pass over a work queue of registered axioms, with four standing rules that every sweep report must satisfy (learned, in part, from early violations the audits later caught):

1. **Registration first:** only registry entries are redeemed; any axiom discovered mid-sweep that is not registered is registered before it is touched.

2. **No silent deletion:** no entry disappears; each is converted to something whose trust status is explicit (theorem with proof, definition, or conditional theorem), and the sweep report records the per-entry mapping axiom-name $\rightarrow$ new form $\rightarrow$ method.
3. **Semantics preserved or explicitly weakened:** the statement of a redeemed axiom may not be quietly changed; if the only honest conversion is a conditionalization (class ① below), the conversion must be visible in the signature itself.
4. **Ledger closure:** the sweep report’s own arithmetic must close against the registry delta — the rule whose early violations became conflicts C5 and C6 (Sec. 5.2).

Six sweeps ran between 2026-08-18 and 2026-08-21 (repository versions v7.94 through the v7.99 working tree), closing $30 + 44 + 31 + 29 + 30 + 32 = 196$ registered axioms — the canonical sweep accounting. Sweep-level facts are summarized in Table 2; the full per-entry records are the sweep reports cited therein [16].

4.2 The five semantic classes

The sweep reports count *closures*. The program’s central bookkeeping decision is that a closure is not a unit of progress; its *semantic content* is. Every closure is therefore re-graded, post hoc and independently of the sweep that produced it (the grading was performed by the P0 reconciliation pass), into one of five classes (plus one excluded category):

- ① **Vacuous placeholder closure** (50 entries, 25.5%). The axiom’s conclusion was `True`; the closure proves it by `trivial` (or `intro`; `trivial`). The theorem now type-checks, is not an axiom, and says nothing. This is honest bookkeeping: the placeholder debt is marked paid *at its true value, zero*.
- ② **$P \rightarrow P$ conditionalization** (90 entries, 45.9%). The axiom asserted P ; the closure is `theorem h : P` with hypothesis (`h : P`) and proof `:= h`. The axiom disappears from the count, the statement remains available to downstream consumers in conditional form, and *no mathematical content has been transferred*. We return to the honesty of this class in Secs. 4.3 and 9.2.
- ③ **Data-to-definition conversion** (17 entries, 8.7%). Axioms asserting numeric facts about physical constants become `defs`: e.g. the critical density, asserted as an axiom with a numeric value, becomes `def rho_c := 3*H0^2/(8*pi*G)` with the numeric value retained in the docstring as data. The axiom footprint shrinks by construction; the “theorem” purchased is definitional, not mathematical.
- ④ **Definitional reconstruction** (24 entries, 12.2%). Axioms whose statements are definitional equalities become theorems proved by `rf1` after the underlying `def` is repaired (constants previously given as bare numbers are redefined by their formulas). Genuine proof work, of exactly zero mathematical depth — but it *is* the correct fix, and it survives `#print axioms`.
- ⑤ **Substantive proof** (13 entries, 6.6%). Real proofs of real statements: identifications via `mathlib` lemmas (e.g. `Real.rpow_pos_of_pos` under an explicitly added physical hypothesis $\alpha' > 0$), algebraic identities by `ring/nlinarith`, existence constructions by `exists_intro`. Thirteen of 196.
- ⑥ **Axiom bundling, excluded from the redemption classes** (2 entries, 1.0%). Two closures restructured a statement as a conjunction of two existing axioms (the Cook–Levin reduction bundle; one analogy-of-science entry). The `axiom` keyword disappears from the declaration count, but both ingredients remain axioms — so the P0 reconciliation

Table 2: The six sweeps and the canonical five-class re-grading of their closures (P0 reconciliation, sec. 4). Column ⑥ counts bundling restructurings excluded from the five-class redemption decomposition.

Sweep	Closed	①	②	③	④	⑤	⑥	Version
sweep1 (2026-08-18)	30	6	1	17	3	2	1	v7.94
sweep2 (2026-08-19)	44	44	–	–	–	–	–	v7.95
sweep3 (2026-08-19)	31	–	–	–	21	10	–	v7.96
sweep4 (2026-08-20)	29	–	27	–	–	1	1	v7.97
sweep5 (2026-08-21)	30	–	30	–	–	–	–	v7.98
sweep6 (2026-08-21)	32	–	32	–	–	–	–	worktree
Total	196	50	90	17	24	13	2	

reports these two *separately* from the five redemption classes: the canonical sentence is “196 closures = five classes summing to 194, plus 2 bundling restructurings”, and no proof-rate or redemption claim anywhere in the program counts them as content.

Two features of Table 2 deserve emphasis. First, the sweeps are *specialized*: sweep2 eliminated the entire placeholder class (all 44 remaining **True**-conclusion axioms in its target set; the class total of 50 includes 6 from sweep1); sweep3 was the definitional pass; sweeps 4–6 were progressively more honest about what remained. Second, the composition drifts *down*, not up: the last two sweeps are 100% $P \rightarrow P$ — not because the methodology degraded, but because the registry queue was drained in order of decreasing semantic availability, and what remains at the bottom is exactly the material that cannot be redeemed honestly without doing or importing mathematics.

4.3 The honesty of $P \rightarrow P$

The $P \rightarrow P$ class deserves its own subsection because it is the methodology’s most easily abused move. The conversion

$$\text{axiom } X : P \quad \longrightarrow \quad \text{theorem } X \text{ (} h : P \text{)} : P := h$$

is a real improvement in three specific senses: the ambient logic no longer *assumes* P ; downstream consumers that cite X must now supply P as a hypothesis, so the debt is propagated instead of hidden; and the statement’s trust status is visible in its own signature. It is a nullity in the sense that matters to a mathematician: no proof of P has been produced, and a proof-rate metric that counted 90 such conversions as 90 proofs would be lying with precision.

The program’s rule, enforced nowhere but in the taxonomy itself, is that $P \rightarrow P$ conversions are only acceptable when (i) the statement is retained verbatim as the conclusion, (ii) the sweep report names the class explicitly, and (iii) the aggregate reporting (the five-class table, never the bare count) keeps the class visible. Sweep6 documented the limiting case honestly: some registered propositions are *false* under the corpus’s stub definitions (e.g. a partition-size identity that holds only for $N = 1$ under its stub, and a positivity statement that fails under a zero-stub), and for those, $P \rightarrow P$ is not merely the cheap conversion but the *only* honest one — asserting the unconditional P was the original defect. Sweep6’s report also discloses an operational constraint: the sweep was executed in an environment without a Lean toolchain, so its conversions carry structural verification (keyword, hypothesis, proof-body, verbatim conclusion) but not compilation — a boundary stated in the report itself rather than papered over.

4.4 Closing the declaration account

The six sweeps, plus non-sweep removals (template axioms deleted from generated-but-committed files during corpus hygiene), produce a monotone evolution of the live declaration count (`axiom` commands in `sylva_formalization/`, statement-level, canonical caliber):

Version	axiom declarations	Δ
v7.94 (registry baseline)	445	—
v7.95	399	−46 (sweep2: 44; non-sweep: 2)
v7.96	368	−31 (sweep3)
v7.97	315	−53 (sweep4: 29; non-sweep: 24)
v7.98	285	−30 (sweep5)
working tree (v7.99)	253	−32 (sweep6)

The account closes exactly: $445 - 166 (\text{sweep}_2..6) - 26 (\text{non-sweep}) = 253$, matching the registry-residual and live-declaration calibers independently (Sec. 5). It is worth stating what this arithmetic does and does not certify: it certifies that *entries* left the axiom ledger by declared, auditable routes; it does not certify that mathematics happened — for that, one needs the five-class column, and there the answer is 13.

5 Three-Caliber Reconciliation

5.1 Why numbers drift

Every aggregate in this program is computed by a script, over a corpus, at a time, through a pattern — change any of the four and the number changes while the library does not. An audit that reconstructed the program’s own reporting history [20] (2026-08-21) found six live conflicts (C1–C6) in which the same quantity carried incompatible values across authoritative-looking documents; two of them (C3, C6) had survived precisely because both sides looked defensible. The conflicts are themselves data: Table 3 lists all six with their resolutions — each resolution is not “the larger number won” but a named caliber plus a reproducible measurement.

The P0 reconciliation pass then did for the whole account what Table 3 did for the conflicts: it fixed one caliber per quantity, measured everything against the working tree, and landed the results as the repository’s canonical register (`framework/p0_reconciliation_report.md` [24]), with the measurements themselves in an appendix so any future audit can re-run them. The rest of this section is the reconciliation, class by class.

5.2 Axioms: three calibers that must all close

Three independent records count “the axioms”: the registry (what was registered), the live declarations (what `axiom` commands the source currently contains), and the sweep ledger (what the redemption process claims to have closed). Each answers a different question, and the program’s accounting rule is that all three must close against each other *or* the discrepancy must itself be a named, measured object:

- **Caliber A (registry entries):** 448 at baseline — primitive 250, definitional 142, placeholder 42, schema 14; 391 unique names, [\(57\)](#) cross-file duplicates, over 116 non-batch files, generated at v7.94. (The registry’s 448 and the v7.94 declaration census’s 445 differ by exactly 3 entries — two documentation-example snippets and one `.backup` archive entry picked up by the generator’s parser; a named caliber difference, not an error in either count. The pre-reconciliation headlines “478” and “350+” were respectively a historical README caliber and an audit-timepoint estimate, both annotated and retired by C1.)

Table 3: The C1–C6 conflict inventory (origin: D1 review sec. 1.3; resolution: P0 reconciliation). Each resolution names the surviving caliber and its correction site.

#	Conflict	Resolution
C1	axiom counts (478 vs. 448 vs. 350+)	resolved: authoritative live count 253 (caliber $A' \equiv B$); 448 is the v7.94 registry baseline (caliber A); 478 was a historical README caliber, 350+ an audit-timepoint estimate — both annotated and retired
C2	Agda postulates (149 vs. 55 vs. none)	resolved: 7 files, 25 top-level blocks, 131 name-level declarations (name caliber is canonical; 55 = string caliber, 149 = planning value; registry’s “no Agda” note corrected)
C3	proof rate (3.91% vs. 99.79%)	resolved: 3.91% authoritative (independent caliber, 2026-08-11); legacy dashboard frozen and marked stale (Sec. 5.4)
C4	batch “sorry contamination” (30.2%)	resolved: string-caliber false positive; code-level count is zero (Sec. 8.3)
C5	sweep3 internal ledger (37 vs. 31)	resolved: 31; the method table’s 37 was a miscount, corrected in the sweep report
C6	“136 redeemed” composition	resolved: canonical 196 with the five-class decomposition (Sec. 4.2); “136/166/198” were double-count or estimate calibers

- **Caliber A' (registry residual): 253** — registry entries that still point at live `axiom` declarations (primitive 239 + schema 14). Detached entries: $448 - 253 = 195 =$ definitional 142 (all cleared) + placeholder 42 (all cleared) + primitive 11.
- **Caliber B (live declarations): 253** — statement-level `axiom` commands under `sylva_formalization/` (252 unique file×name pairs; one name declared twice in a single file; 214 unique names). $A' \equiv B$ is the reconciliation’s strongest single check: two records produced by different processes (registration vs. measurement) agree entry by entry.
- **Caliber B' (whole-repo declarations): 284** — adds [31](#) statements outside the audited scope (academic courseware 15, archive 11, released src snapshots 4, papers 1). Quoted only when the question is “what does the whole repository assert?”, never as a library headline.
- **Caliber C (sweep accounting): 196** — closures claimed by sweeps 1–6 ($30 + 44 + 31 + 29 + 30 + 32$), re-graded into the five semantic classes of Sec. 4.2.

The calibers close through two identities. First, *baseline identity*: caliber A (448) minus the detached-in-sweep-and-other-processes entries equals the residual: registry $448 \rightarrow 253$ residual. Second, *evolution identity* (Sec. 4.4): the declaration census moved $445 \rightarrow 399 \rightarrow 368 \rightarrow 315 \rightarrow 285 \rightarrow 253$ across versions v7.94→working tree, and $445 - 166$ (sweeps_{2..6}) – 26 (non-sweep removals) = 253. The two “headline” numbers that this reconciliation retired — “198 closures” and “598→568 declarations” — failed for instructive reasons: the former double-counted 2 entries already inside sweep1’s 30 (the berryPhase pair, committed at v7.94 before the sweep reports were written), and the latter is reproducible under *no* caliber the audit could construct (sweep5’s actual effect was $315 \rightarrow 285$) — a fabricated-precision artifact, now corrected in the sweep report itself.

5.3 Agda: three calibers for one track

The Agda side inherited the same disease in miniature: the governance documents said “~149 postulates” (a planning estimate from before the files existed in their current shape), a string scan said 55, and the registry’s metadata said there were no Agda files at all. The P0 register fixed all three: 7 files, 25 top-level `postulate` blocks, and **131** name-level declarations (the canonical caliber), with the per-module split Quantum 53 / Spectrum 36 / Complete 15 / CNFCategory 12 / Order 12 / Cauchy 2 / Field 1 summing to 131. The string caliber’s 55 counts the *keyword occurrences*; the planning value’s 149 counts an intention. Neither measures the library. The Agda redemption (131 postulates to be replaced by standard-library proofs) remains open and is the largest single visible debt line in the program’s roadmap — deliberately not swept into any closure number.

5.4 Proof rate: the 25× disagreement

The program’s metric history is the paper’s cautionary core. An early dashboard reported **<88.08%** “proved” under a hand-computed caliber; successive revisions reported **<99.5%**, **<99.9%**, then **<100%** — each step technically defensible under its own definition and collectively absurd. Two artifacts then disagreed by 25×:

- **Authoritative (independent caliber): 3.91%** = $22,328 \div 571,611$, computed 2026-08-11 by the reference implementation (`scripts/compute_proof_rate.py`, `count_sorry_no_comments`): handcrafted theorems only, `sorry` counted at code level with comments stripped, and axiom-dependent theorems *excluded from the numerator* — i.e. the metric respects the difference between a proof and a promise. The batch corpus is reported on its own line (0% under the same caliber), never merged.
- **Stale snapshot: 99.79%** (with 100.00% batch and 79.70% legacy rows) — a hand-patched dashboard frozen 2026-08-12, whose denominator (546,356) and axiom count (553 vs. the independent caliber’s 586) differ from the authoritative caliber. The P0 register’s disposition: the snapshot stays in the repository as historical evidence, marked stale with a warning banner, superseded by the 3.91% caliber.

The reconciliation does not ask which number is “right”; it asks what question each number answers, and keeps only the one whose question is the program’s actual question: *of the theorems we claim, how many are proved?* — answer, at the time of writing: 22,328 of 571,611 handcrafted, and none of the batch corpus. The caliber standard that emerged (string / code / CI-pattern levels, with the code level normative) is formalized in Sec. 8.2.

5.5 The reconciliation as a first-class artifact

Three practices made the reconciliation itself trustworthy, and all three are transferable:

1. **One canonical register per quantity family.** The P0 report is designated the sole authoritative source for axiom/Agda/proof-rate/sweep numbers; downstream documents either quote it or are wrong. (The register explicitly lists the downstream documents that still carry pre-reconciliation values and whose fixes belong to other workstreams — errata surface, not silent fixes.)
2. **Measurements in the appendix.** Every canonical number in the register is reproducible by a one-line command in the report’s appendix (registry Python reconciliation, declaration censuses per version, Agda block parsing, the CI honesty gate). An auditor should never need to trust the register’s arithmetic.

Level	Statement	Audit verdict
L5	$\alpha^{-1} \approx n_{\text{CS}} = 137$	CLAIM: numerical pipeline; blind-registry freeze; falsifiability clause on file
L4	spectral bridge: causal network $\rightarrow$ CS level (axiom <code>causalNetworkChernSimonsLevel</code>)	AXIOM, registry class <i>schema</i> ; physics derivation, not provable in mathlib
L3	$n_{\text{CS}} \in \mathbb{Z}$ (<code>chernSimonsLevelInteger</code>)	compiled, but degenerate: placeholder def $\equiv 137$ makes integrality a <code>rfl</code> (honestly labeled)
L2	stratified integrality T3: integer weights $\Rightarrow n_{\text{strat}} \in \mathbb{Z}$	THEOREM (this section): compiled, zero <code>sorry</code> , standard axiom trio, mathlib-delegated
L1	stratified-formula semantic cluster (T0–T5, weighted sums, concentration, sector decomposition)	compiled; closes at the sum-structure layer
L0	$U(1) = \mathbb{R}/2\pi\mathbb{Z}$ quotient-group structure	compiled from mathlib quotient instances (v7.94-era P0 fix)

Table 4: The Chern–Simons chain, L5 (physics) to L0 (arithmetic). Census per `framework/chern_simons_theorem_report.md` [18] (v7.98 baseline). The upgraded level is L2; the two blocking gaps are L4 (schema axiom) and L3’s placeholder degeneration.

3. **Caliber names in the citations.** When this paper quotes 448, 253, 196, 131, or 3.91%, the surrounding sentence names the caliber (registry / declaration / sweep / name-level / independent). The parenthetical numbers in red brackets ((...)) are the remaining verified baselines outside the P0 register’s scope — same discipline, one tier down.

6 Case Study I: The Chern–Simons Chain and Theorem T3

6.1 The claim and the chain

The program’s flagship physical claim (CLAIM 1 in the registry) identifies the fine-structure constant’s inverse with a Chern–Simons level number, $\alpha^{-1} \approx n_{\text{CS}} = 137$, and froze that prediction in a blind registry before the comparison was performed. Its falsifiability clause is explicit: a first-principles prediction of α^{-1} accurate to ± 0.01 without loop-integer self-tuning would refute the identification. The claim’s support is a numerical pipeline, not a proof; the governance question is where, along the chain of statements between the physics and the arithmetic, machine-checked mathematics actually reaches.

Table 4 answers. The chain from the physical identity down to quotient-group arithmetic decomposes into six levels (L5–L0), each an independently checkable statement. The audit census found: two levels are genuinely compiled mathematics (L0, L1); one level is a compiled but *degenerate* theorem (L3, true only because a placeholder definition hard-codes the integer); one level is an unredeemed schema axiom (L4); the top level is exactly the CLAIM of the blind registry (L5); and the remaining level, L2 — integrality of the stratified Chern number under integer weights — is the deepest link whose proof closes without touching any unredeemed axiom.

The audit also censused the side chains that hang off the main one: knot-theoretic identifiers (Witten–Jones type, (15) `True := trivial` placeholders), instanton indices (AS index, 2 standing axioms with conditionally compiled theorems), quantum Hall identifiers (TKNN-type, (21) vacuous placeholders), and the Chern-number family side files (axiom carriers, all graded CLAIM). In aggregate the side chains carry (80+) vacuous entries plus 2 axioms — each an independent mathematical infrastructure project (gap G4 below). None of them supports CLAIM 1;

the registry keeps them visible precisely so that no reader mistakes their existence for evidence.

6.2 The upgrade: what was proved, and how it is labeled

The CLAIM→THEOREM upgrade was performed at L2, on the theorem

```
theorem stratifiedChernNumber_int_weights (n v :  $\mathbb{Z}^{\times k}$ ) :
  ↑ (stratifiedChernNumberInt n v) = stratifiedChernNumber ↑ n ↑ v — i.e.  $\uparrow \sum_{\alpha} n_{\alpha} v_{\alpha} = \sum_{\alpha} \uparrow (n_{\alpha}) \uparrow (v_{\alpha})$ .
```

The selection was a governance decision, not a mathematical boast, and the registered rationale has three parts. *First, position:* T3 is the deepest link at which the statement closes without touching any unredeemed axiom — the PFE source paper itself concedes that its integrality lemma (6.2) had only a numerical pipeline, and that all its theorems close at the sum-structure layer. *Second, evidence quality:* T3 was compiled for real on 2026-08-10 (Lean 4.29.0, mathlib4 pinned at 8a178386, exit 0, 6.7s), with `#print axioms` returning only the standard trio [`propext`, `Classical.choice`, `Quot.sound`] on all six theorems of the module; the log is archived in the panel records [23]. *Third, delegation pattern:* the proof discharges the $\mathbb{Z} \rightarrow \mathbb{R}$ cast through two named mathlib lemmas — `Int.cast_sum` (casts commute with finite sums) and `Int.cast_mul` (casts commute with multiplication), applied under `Finset.sum_congr` — following the same named-lemma discipline as the Dedekind case study (Sec. 7.2).

The proof is three rewrites long. That is not an embarrassment; it is the point. The registration’s value is the *label*, not the difficulty: the row in the claim registry now says THEOREM and links, conjunct by conjunct, to (i) the compile log, (ii) the axiom-trio certificate, (iii) the named mathlib delegation, and (iv) an explicit non-degeneracy note distinguishing T3 from the L3 placeholder theorem above it. A companion strengthening round added the $\exists$ -form corollary T3’ (matching the logical shape of `chernSimonsLevelInteger` so the L2/L3 interface is fixed), plus T6/T7 (integer-sector 137-degeneracy and concentrated-weights forms) — added under a zero-new-vocabulary discipline, with the honest disclosure that the strengthening round executed in a no-toolchain environment, so the THEOREM grade rests on the compiled T3 alone; T3’/T6/T7 carry an awaits-independent-compile note (one `lake env lean` command, single file, pure mathlib dependencies).

6.3 What T3 is not: gaps and the rollback incident

An internal mathematics-physics referee (the D1 review’s R2) graded the upgraded statement bluntly, and the grading is preserved in the paper: T3 is a $\mathbb{Z}$ -ring closure identity — the cast homomorphism commuting with sums and products — *not* a topological integrality theorem. The physically meaningful integrality (Chern–Weil: $\langle c_1(E), [M] \rangle \in \mathbb{Z}$, carried by a cohomological integral) is nowhere in it; with integer weights the integrality holds by construction. The corrected formulation, which this paper adopts verbatim: T3 formalizes the *degeneration consistency* of the stratified formula on the integer-weight sector and exhibits the precise sufficient condition (integer weights) under which integrality holds; real-weight integrality — the physical target — remains an open gap. Any sentence that lets the reader hear “T3 supports $\alpha^{-1} = 137$ ” is a caliber error of the type this paper exists to prevent; the blind-registry row keeps its CLAIM grade.

The case study also records an incident that no clean retrospective would invent. The upgrade’s registration (a THEOREM row in `proof_status.md` [25]) and the T3’/T6/T7 additions were performed by one session while a *parallel* redemption sweep was running; the sweep session, isolating what it believed were foreign edits, rolled both back with `git checkout`. The changes existed, then did not — and the audit layer caught the disappearance itself: a scheduled evidence check found the registration missing, timestamps and `git status` pinpointed the rollback, and

Gap	Content	Why it stands
G1	Chern–Weil theory (characteristic classes, de Rham cohomology, first-Chern integrality)	mathlib has none of it; FirstChernNumber_Integrality , ChernNumberInteger , TKNN_Formula remain axioms
G2	spectral (causalNetworkChernSimonsLevel) bridge	schema axiom; a physics derivation, not a pure-mathematics statement
G3	placeholder degeneration of L3 (chernSimonsLevel $\equiv$ 137)	integrality at L3 is rfl-true by hard-coding; T2/T6 pin the compatibility interface while the placeholder stands
G4	side chains: knot/AS-index/QHE identifiers, <80+> vacuous + 2 axioms	each needs independent mathematical infrastructure before upgrade

Table 5: Registered gaps above and beside the upgraded link. The upgrade is honest only together with this table.

the sweep report’s own log (“+60 lines ... belongs to another session ... rolled back”) confirmed the mechanism. Under the program’s standard, *an uncommitted registration is a non-event*: the upgrade was re-applied and committed with the next version. The incident produced a standing rule — registration files are serial-exclusive, and no session rolls back another session’s registry rows — and it is reproduced here because it is the honesty layer working as designed, not as advertised.

Read as mathematics, then, the T3 upgrade is one small cast-rearrangement lemma sitting under two admitted gaps and a CLAIM it does not touch. Read as governance, it is the full lifecycle: claim identification, chain census, deepest-provable selection, compile evidence, delegation labeling, gap accounting, and a rollback the audit layer itself detected. The story is in the inventory audit trails, not the mathematics. The governance contribution is the label: within this corpus, “THEOREM” now means exactly “compiled, zero-**sorry**, standard-axiom-trio, non-degenerate, with a named proof delegation”, and the registry row points at the evidence for each conjunct.

7 Case Study II: The Dedekind Delegation Layer

7.1 The course chain and the delegation pattern

The second case study is deliberately the opposite of the first. The Chern–Simons chain (Sec. 6) is a physics claim whose deepest provable layer had to be *found* inside a dependency chain of placeholders; the Dedekind module is a textbook chapter of commutative algebra whose proofs were never in doubt and whose only governance question is *attribution*: when every theorem in a file is discharged by a named mathlib lemma, what may the library claim about itself?

CourseChain/Dedekind.lean is the second segment of a course chain (first segment: $\mathbb{Z}_p/\mathbb{Q}_p$, v7.95; this segment: Dirichlet theory background plus Dedekind domains, v7.96) that formalizes the algebraic-number-theory curriculum in Lean 4. It states the foundational theorems of Dedekind domain theory — a PID is a Dedekind domain; $\mathbb{Z}$ is a Dedekind domain; ideal divisibility is reverse containment; unique factorization of ideals; prime ideals and residue norms; finiteness of primes over a given ideal — and proves each by *direct delegation*: **exact/** $:=$ to a named mathlib lemma or instance whose signature was verified against the pinned mathlib revision (8a178386) in the module header. The design rules are explicit: no unfinished proofs, no new postulates, no trivial placeholders, and *only* theorems whose mathlib counterpart could be located and signature-verified — “no guessing of lemma names.” The textbook anchor (Neukirch, *Algebraic Number Theory*, Chapters I and III.1) is cited in the header so that the curriculum

Table 6: Dedekind delegation table (CourseChain/Dedekind.lean): every theorem discharges to a named mathlib entity; the flagship row carries the full instance chain. Zero `sorry`, zero `axiom`, zero `trivial` in the module.

Course-chain theorem	mathlib delegate	Delegate kind
<code>pid_isDedekindDomain</code>	<code>IsPrincipalIdealRing.isDedekindDomain</code>	instance
<code>int_isDedekindDomain</code>	(as above, via <code>inferInstance</code>)	instance
<code>ideal_dvd_iff_le</code>	<code>Ideal.dvd_iff_le</code>	lemma
<code>ideal_uniqueFactorizationMonoid</code>	<code>Ideal.uniqueFactorizationMonoid</code> (<code>Ideal.Basic L432–447</code>)	instance
<code>ideal_prod_normalizedFactors</code>	<code>Ideal.prod_normalizedFactors_eq_self</code>	lemma
<code>ideal_prime_iff</code>	<code>Ideal.prime_iff_isPrime</code>	iff-lemma
<code>ideal_absNorm et al.</code>	<code>Ideal.absNorm</code> , <code>Ideal.absNorm_span_singleton</code>	def/lemma
<code>primesOver_finite</code>	<code>IsDedekindDomain.primesOver_finite</code>	lemma

intent is checkable against the literature.

Table 6 is the attribution table the module maintains, normalized to paper form (per-lemma mathlib anchors recorded in the source; the flagship instance chain shown).

7.2 Zero new mathematics, and why it is registered anyway

The internal peer review of this module was blunt (journal_submission_d1_review, sec 2.1 [20]): “all theorems are delegation wrappers; zero new mathematics.” Under a mathematics-first venue that is a rejection. Under the governance lens of this paper it is the cleanest possible demonstration of what the claim tiers are *for*:

- The registry entry is `THEOREM`, not `THEOREM*`: the proof (compiled delegation to a proved mathlib instance chain) is machine-checked within the corpus, complete, and carries zero local obligations — `inferInstance` to `Ideal.uniqueFactorizationMonoid` (whose own derivation in mathlib runs through ideal invertibility and `WfDvdMonoid` to `UniqueFactorizationMonoid`), with the pinned-commit source lines recorded. The three-part registration rationale (statement’s role in the curriculum; proof chain with named delegates; compile-status basis) is the same format as T3’s.
- `THEOREM*` is reserved for the other delegation pattern: mathematics proved *elsewhere* and formalized *elsewhere*, which the corpus only cites — the registry’s Fermat’s Last Theorem row (`THEOREM*`, “mathlib-provided, not in this repository”) is the canonical example, with the repository quoting mathlib’s scale for it (`<12000` lines of FLT-specific development atop a `<270000`-theorem prerequisite stack, per the registration note).
- What the module *adds* is not mathematics but a verified attribution mechanism: signature-level anchoring of every course-chain statement to a pinned mathlib revision, a textbook correspondence, and a compile-clean module boundary. When the course chain later grows segments whose proofs are only partly in mathlib, the delegation table is precisely the surface on which “local work vs. borrowed work” will be visible — the same surface the five-class taxonomy provides for axioms.

7.3 T3 vs. Dedekind: two ends of one scale

Set side by side, the two case studies calibrate the whole program. Both registrations are THEOREM; both rest on `#print axioms`-clean compilations against the same pinned mathlib revision; both use *named* delegation (`Int.cast_sum/Int.cast_mul` for T3, `inferInstance/exact` for Dedekind). They differ in exactly one dimension: where the local contribution sits. Dedekind’s contribution is the attribution layer itself (statements curated, signatures verified, zero local proof content); T3’s contribution is a small but real assembly step — the integrality of a stratified sum is not a mathlib lemma, and discharging it required combining three named lemmas in a proof whose shape was not copied from anywhere. The scale between “pure delegation” and “assembly” is the honest axis along which the corpus’s non-placeholder content must be presented; pretending either end is “a new theorem” is exactly the failure mode F1 that the governance program exists to prevent. The course chain continues (v7.95: $\mathbb{Z}_p/\mathbb{Q}_p$; v7.96: Dirichlet background + Dedekind), and each future segment will be graded on this axis at registration time.

8 Honesty Governance

8.1 Claim tiers and falsifiability

The governance layer begins with a vocabulary problem: the corpus used “formalized” for statements ranging from textbook theorems with complete machine-checked proofs to naked `axiom` placeholders. The registry therefore fixes a four-tier vocabulary (`proof_status.md` sec 1 [25]):

Tier	Meaning
THEOREM	machine-checked proof within the corpus (or complete delegation to a checked external proof), core free of <code>postulate/axiom</code> ; certificate on file
THEOREM*	mathematical content proved and formalized <i>elsewhere</i> ; the corpus delegates (e.g. FLT via mathlib)
CLAIM	reasoning, numerics, or citations exist, but no independent formalization; falsifiability criteria mandatory
CONJECTURE	speculative, logically coherent, falsifiable

Two design decisions matter for the rest of this section. First, the tiers are *vocabulary for status, not for value*: a CLAIM is not an embarrassment, it is a precisely delimited obligation — every CLAIM in the registry carries a falsifiability clause (v7.24 discipline), e.g. for CLAIM 1: a first-principles prediction of α^{-1} accurate to ± 0.01 without loop-integer self-tuning. Second, tier assignments are *evidence-linked*: each THEOREM row names the source file, theorem, compile evidence, and proof chain (Secs. 6 and 7.2 showed the two hardest cases), and the “key distinction” printed under the registry table is the program’s motto: “*formalized*” $\neq$ “*proved*” — *an axiom is proof deferred to the future*.

8.2 The honest proof-rate metric

The registry’s second pillar is a metric that refuses to count deferred proofs as progress. The *honest proof rate* is

honest rate = (theorems derived with zero `axioms`, zero `postulates`, zero `sorry`s) $\div$
 (all theorems the corpus claims),
 with handcrafted and batch content **reported separately** — merging them into one
 “total rate” is prohibited by the caliber standard.

Under this metric the corpus’s handcrafted tier scores 3.91% (22,328 of 571,611, independent dashboard caliber, 2026-08-11), and the batch tier is listed separately at 0% by construction. The caliber history that led here is itself part of the finding: an early 88.08% line counted batch trivialities as proven theorems; successive recalibrations reported 99.5%, 99.9%, then 100% (the “zero `sorry` per module” caliber that ignored `axioms` entirely) — all now formally deprecated — before the current definition landed. A number this mutable is not a measurement; binding it to a named caliber (Sec. B) is what turns it back into one. The normative reference implementation (`compute_proof_rate.py`, `count_sorry_no_comments`) counts code-level `sorry`s only, comments stripped — the same string/code distinction that the next subsection dramatizes.

8.3 The sorry incident: a 30.2% contamination that wasn’t

In 2026-08-21, CI slimming analysis [19] reported that [⟨30199⟩](#) of [⟨119859⟩](#) batch files (30.2%) “contained `sorry`”, apparently contradicting the batch generator’s zero-sorry design. The contamination audit (full population, not sampling) reversed the alarm:

Pattern	Files	Share	Nature
header negation statement	36,198	99.997%	fixed boilerplate Actual Lean 4 proofs --- no sorry, no True := trivial hit by substring search
name-level occurrence + trivial body	1	0.003%	SYLVA_ProvenLayer2.lean: theorem <i>name</i> contains “sorry”, and 8 of its 10 theorems are True := trivial
code-level <code>sorry</code> (<code>^\s*sorry\b</code> , comments stripped)	0	—	the CI pattern’s actual check

The single genuine offender was quarantined (below, [26]), and the string-caliber alarm was re-classified as a *caliber-level confusion*, not a contamination event — the CI slimming report’s “hypothesis violated” note was corrected accordingly. The audit codified the three-level caliber standard that the reconciliation chapter applies to every number: **string level** (substring search — never a valid basis for honesty claims), **code level** (statement-level pattern, comments stripped — the reference caliber), **CI-pattern level** (the exact regular expression the honesty gate uses). A side observation with governance weight: under the *string* caliber, filtering for files that “look honest” would have deleted a [⟨14%⟩](#) slice of the Analysis domain — caliber errors are not cosmetic, they misallocate deletion decisions.

8.4 Quarantine, not deletion

The response to the one contaminated module is the repository’s template for handling genuinely bad content: `SYLVA_ProvenLayer2.lean` was moved to `releases/quarantine/` with a machine-readable manifest (origin path, SHA-256, reason list, exclusion scope), the batch census was updated from [⟨119859⟩](#) to [⟨119858⟩](#) files, and its 10 template theorems were removed from the metric’s numerator and denominator alike. Nothing was silently deleted; the no-silent-deletion rule (below) applies with special force to content whose existence is embarrassing. The quarantine directory is the batch corpus’s equivalent of the deleted-claims ledger: a place where failures remain visible, attributable, and countable.

8.5 The honesty CI and its explicit boundary

The governance layer is machine-checked. `verify_honest_repo.py --ci` gates every round: a forbidden-term scan with deletion-context allowance (so that *deleting* a fabricated claim is never itself flagged), the required honesty markers (AI-assisted-generated banners on every report), the deleted-claims ledger (`data/deleted_fabricated_claims.json` [22], [⟨15⟩](#) entries

from the counting-geometry fabrication audit, well above the gate’s 10-entry minimum), BibTeX integrity checks, and new-paper presence checks. It passes in seconds (exit 0), every round.

The gate’s *boundary is stated in the gate itself*: it checks the honesty layer — markers, ledgers, manifests, caliber-discipline artifacts — and deliberately does not check Lean proof content (no `lake build`; the full build exceeds the audit machines’ `<4GB`)-class memory, and the Agda side has never compiled in CI for the same reason). Conflating “honesty CI green” with “library correct” would be a caliber error of exactly the kind this paper catalogs; the Lean side is instead pinned (mathlib4 `8a178386`, toolchain Lean 4.29.0, lakefile + manifest committed) and carries independent per-module compile logs, while the batch generator is reproducible from its manifest (`<119831`) entries, SHA-256 sampling `<500`, zero mismatches). The remaining gap — containerized, artifact- publishing reproducibility — is registered as future work (Sec. 9.4).

9 Discussion and Limitations

9.1 Caliber discipline as the central thesis

The paper’s transferable claim is not the specific numbers of one library but a methodological proposition: in a formal library that mixes human and machine authorship, a proof-progress metric is meaningful only as a (number, caliber) pair, with the caliber declared, machine-checkable, and enforced by the same CI that enforces the rest of the honesty layer. The program’s two self-caught incidents are the evidence.

The 30.2% “sorry contamination” alarm [27] was produced by a string-level scan over the batch corpus: 36,199 of 119,859 files “contained sorry”. A full-population re-audit under the code-level caliber found the true count to be zero: 36,198 of the 36,199 hits were the fixed header comment *asserting the absence* of placeholders, and the one remaining hit was a theorem whose *name* contains the word. The alarm had been measuring comments, not proofs — and the operational consequence cuts both ways: a filter built on the string caliber would have quarantined roughly a third of a clean corpus. The dashboard history tells the same story in percentages: the headline rate climbed 88.08% → 99.5% → 100% across successive documents before the honest re-definition dropped it to 3.91%. Every intermediate value was defensible under its own (undeclared) caliber; the sequence is absurd only against a fixed one.

The redemption ledger closes the argument. Decomposed into the five-class taxonomy [24], the 196 closures are: 50 vacuous placeholder closures (25.5%), 90 $P \rightarrow P$ conditionalizations (45.9%), 17 data-to-definition conversions (8.7%), 24 definitional reconstructions (12.2%), and 13 substantive proofs (6.6%), plus 2 bundling restructurings excluded from the content count. Each of the two large classes is individually defensible — a vacuous placeholder was never mathematics, and a $P \rightarrow P$ conditionalization is a conservative extension that at least makes the dependence explicit in the statement’s type — yet together they are proxy-clean, and 140/196 (71.4%) of closures carry no proof content at all. A redemption program optimized on any aggregate metric would drift exactly here: conditionalization removes an `axiom` declaration, improves every census, and transfers nothing. A skeptic who suspects a proof-rate metric of being gamed now has a concrete audit program: decompose the numerator into semantic classes, declare the caliber of every count, and reconcile ledger, process, and live-code totals. That is the paper’s export.

9.2 Limitations

This section states, rather than defends, what the program has not achieved.

1. **Most closures transfer no mathematics.** The canonical decomposition is unflattering by design: 140 of 196 redemptions (71.4% — the 90 $P \rightarrow P$ conditionalizations and 50 vacuous closures) add no proof content, and the 13 substantive proofs (6.6%) are themselves

shallow — arithmetic rewrites, existence witnesses, and single-lemma mathlib discharges. The declaration census improving 445→253 is a record of bookkeeping progress, not of mathematical progress, and this paper deliberately refuses the aggregate “196 redeemed” reading.

2. **$P \rightarrow P$ conditionalization is a loophole the taxonomy only exposes.** The conversion `axiom X : P → theorem X (h:P) : P := h` is logically conservative and audit-proof: it removes an axiom, changes no theorem, and cannot be faulted under any declaration-level metric. A library could convert its entire axiom debt this way and truthfully report “zero axioms” while having verified nothing new. The five-class taxonomy makes such a program *visible*; it does not yet *prevent* it — the CI gate audits the honesty of reporting, not the depth of content.
3. **The conservativity claims are not yet metatheorems.** That the three conversion families (conditionalization, vacuous closure, definitional reconstruction) are conservative extensions is asserted from inspection of the sweep records, not proved. Until the small metatheory is formalized (the registered P1-6 program), the taxonomy’s semantics rest on exactly the kind of undischarged trust this paper is about retiring.
4. **The residual debt is large and partly permanent.** 253 axioms remain (239 primitive, 14 schema), including the spectral-bridge axiom of Case Study I, which is physics rather than mathematics and may never be “redeemed” in the sweep sense. The Agda track’s 131 postulates have never compiled in CI (the toolchain OOMs on the audit machines). The honest proof rate of 3.91% is itself the statement that the corpus is at an early stage of genuine coverage.
5. **Verification boundaries.** The honesty CI checks the honesty layer (markers, ledgers, manifests, caliber artifacts); it does not run `lake build`. Sweep6 executed without a Lean toolchain (structural verification only); the T3’/T6/T7 strengthening of Case Study I awaits its independent compile. Reproduction is command-level and version-pinned, not yet containerized (P1-8).
6. **Single corpus, no baseline benchmark.** The methodology is evaluated on one library — this one. The comparison against naive `grep` and mathlib’s linters is qualitative (Sec. 2); a measured detection/false-positive baseline comparison is future work. All figures are anchored to the v7.99 working tree and will drift; the caliber discipline, not the numbers, is what transfers.

9.3 Anticipated objections

Four objections the internal review pre-registered [20], with the program’s standing answers:

1. “*The corpus is AI-generated; why trust any of it?*” That is the premise, not the objection. The governance layer exists because AI-assisted generation at this scale cannot be trusted through review capacity. Trust is replaced by verifiability: every headline number is a command away (Appendix A), every claim row links its evidence, and every artifact carries its provenance markers.
2. “*The remaining axioms are non-standard.*” They are registered, class-labeled, and deliberately visible; the program nowhere claims they are proved. The reconciliation exists so that no document can quietly re-count them.
3. “*The 119,859-module batch tier poisons the metrics.*” The caliber standard prohibits merging tiers: the batch rate is reported separately (0% honest rate by construction), the

one contaminated module is quarantined, and the 30.2% alarm turned out to be a caliber artifact — which is precisely why tiers and calibers are enforced in CI.

4. *“Your own numbers disagreed; why believe the current ones?”* The drift is the finding. Six documented conflicts (C1–C6) were resolved through a named-caliber reconciliation whose every step is reproducible; the errata are surface-visible, never silent. A numbers system that can exhibit its own corrections is the strongest thing an auditor can be offered.

9.4 Future work

The registered program, in order: (i) the metatheorem track — formalizing the conservativity of the three conversion families, which would convert this paper’s taxonomy from an audit practice into a method with proved properties; (ii) evaluation of the residual: the 239 primitive and 14 schema axioms need a grading that distinguishes permanent physics postulates from redeemable mathematical debt, and the Agda 131 require a machine that can compile them; (iii) a non-delegation theorem target on the Chern–Simons chain (the registered candidate: a weight-group decomposition theorem generalizing T6’s integer-sector invariance), the first prospective THEOREM that would be neither a wrapper nor a cast identity; (iv) containerized reproducibility (Dockerfile, pinned toolchain, expected-output snapshots), closing the P1-8 gap; (v) venue checkpoints per the journal-matching matrix [21]: ITP 2027 as the mid-way conference target, CPP 2028 as the main line, JAR as the journal target this draft is shaped for. The blind registry keeps its own schedule: $\alpha^{-1} = 137$ stays a frozen prediction until a first-principles computation or its refutation arrives.

9.5 Ethics and transparency

The library and this paper were produced with AI assistance under human tasking and review. The repository’s transparency conventions (AI-assistance markers on every artifact, deleted-claims ledger, no-silent-deletion rule, CI-enforced honesty markers) apply to this manuscript: canonical numbers, bracketed baselines, draft status, and pending work items are visible by construction, and the supplementary material includes the build log of the exact $\text{\LaTeX}$ compile that produced the accompanying PDF. One decision deserves explicit statement: the program does *not* delete the fabricated or vacuous material it finds — it labels, quarantines, or converts it, leaving the evidence in place. For an AI-assisted corpus, an audit trail that survives is worth more than a clean but unverifiable history.

10 Conclusion

We presented the axiom-governance methodology of TOE-SYLVA, an AI-assisted formal library that scaled past its own review capacity, and the honesty layer that now disciplines it. The methodology has four moving parts, each independently useful and together closing the loop:

1. **A registry-first audit:** every `axiom` declaration in the non-batch corpus is registered with a semantic class (448 entries at baseline: primitive 250, definitional 142, placeholder 42, schema 14), so that debt is explicit before it is addressed.
2. **Redemption sweeps with semantic re-grading:** six sweeps closed 196 entries ($30 + 44 + 31 + 29 + 30 + 32$), and the P0 reconciliation re-graded them into five classes — vacuous 50, $P \rightarrow P$ conditionalization 90, data-to-definition 17, definitional reconstruction 24, substantive 13, plus 2 bundling restructurings excluded from the content count. The honest headline is 13.

3. **Three-caliber reconciliation:** registry (448), live-declaration (253), and sweep-process (196) accounts now close identically ($445 - 166 - 26 = 253$ and $448 - 253 = 195 = 142 + 42 + 11$), the Agda side is pinned at 7 files / 25 blocks / 131 name-level postulates, and the authoritative proof rate is 3.91% (22,328/571,611, handcrafted; batch reported separately).
4. **An honesty layer:** claim tiers with falsifiability criteria, a caliber-discipline standard, a quarantine protocol, and a CI gate that fails on silent deletions and missing markers.

Two case studies showed the lifecycle end to end: a CLAIM $\rightarrow$ THEOREM upgrade along a Chern–Simons chain whose registered theorem is deliberately labeled as what it is — a `mathlib`-delegated cast-summation identity, not topological integrality — and a Dedekind-domain segment that is honest about contributing zero new mathematics while validating the attribution mechanism.

For the growing population of AI-generated formal content, we claim one general lesson. Every number this paper reports — including the unflattering ones — is bound to a declared, reproducible caliber. That binding, not any particular percentage, is what makes a proof-rate metric trustworthy; and the cheapest time to install it is before the corpus, and the metrics, have a history worth laundering.

References

- [1] Autoformalization in the wild. arXiv:2502.12065. gap warning on real mathematical definitions; TODO(P1-5) formal entry.
- [2] Autoformalization of LLM-generated outputs. arXiv:2511.11829. formal-verification path for LLM outputs; TODO(P1-5) formal entry.
- [3] Lean Atlas. arXiv:2604.16347. semantic hallucination warning: type-checking pass $\neq$ faithful proposition; TODO(P1-5) formal entry.
- [4] Lean Copilot: an LLM copilot for the Lean proof assistant. arXiv:2404.12534. interactive LLM proof assistance; TODO(P1-5) formal entry.
- [5] Lean Workbook. arXiv:2406.03847. NL $\rightarrow$ Lean problem-set construction; TODO(P1-5) formal entry.
- [6] LLM-driven user-intent formalization. arXiv:2406.09757. evaluating whether propositions faithfully express intent; TODO(P1-5) formal entry.
- [7] MASA: multi-agent autoformalization framework. arXiv:2510.08988. role-division + consistency-check architecture; TODO(P1-5) formal entry.
- [8] Metamath. <https://us.metamath.org/>. TODO(P1-5): axiom-economy tradition citations.
- [9] Paraphrase-induced failures in proof autoformalization. arXiv:2604.23135. robustness of frontier models to paraphrase; TODO(P1-5) formal entry.
- [10] Process-driven autoformalization (formL4). arXiv:2406.01940. process-driven Lean 4 autoformalization benchmark; TODO(P1-5) formal entry.
- [11] RLMEval: evaluation for real-world Lean projects. arXiv:2510.25427. research-grade benchmarking stance of this program; TODO(P1-5) formal entry.

- [12] Robustness of proof autoformalization. arXiv:2606.14867. non-canonical real proof texts; TODO(P1-5) formal entry.
- [13] The Isabelle/HOL proof assistant. <https://isabelle.in.tum.de/>. TODO(P1-5): include AFP review-process citations.
- [14] The Lean 4 theorem prover and programming language. <https://leanprover.github.io/>. TODO(P1-5): upgrade to formal bibliographic entry.
- [15] The mathlib4 Community. mathlib4: the Lean 4 mathematical library. <https://github.com/leanprover-community/mathlib4>. Pinned at commit 8a178386 in this library; TODO(P1-5): upgrade to formal entry.
- [16] TOE-SYLVA Formalization Governance Group. Axiom reduction sweeps 1–6: Per-entry redemption reports. Technical report, TOE-SYLVA repository, 2026. `framework/axiom_reduction_sweep{1..6}_report.md`; canonical totals per P0: sweep1-4 = 134, sweep1-5 = 164, sweep1-6 = 196 (30+44+31+29+30+32); sweep3 method-table erratum (37→31) and sweep5 report errata included.
- [17] TOE-SYLVA Formalization Governance Group. Axiom registry v2.0 (machine-readable). Technical report, TOE-SYLVA repository, 2026. `framework/axiom_registry.json`; 448 entries (primitive 250 / definitional 142 / placeholder 42 / schema 14) over 116 non-batch files, generated at v7.94; canonical per P0.
- [18] TOE-SYLVA Formalization Governance Group. Chern–simons chain theorem report (t3 lifecycle case study). Technical report, TOE-SYLVA repository, 2026. `framework/chern_simons_theorem_report.md`; L0–L5 chain census, side-chain inventory, T3 proof chain, T3'/T6/T7 disclosure, gaps G1–G4.
- [19] TOE-SYLVA Formalization Governance Group. Ci slimming phase 1 report (batch-corpus context and generator reproducibility). Technical report, TOE-SYLVA repository, 2026. `framework/ci_slimming_phase1_report.md`; manifest 119,831 entries, SHA-256 sampling 500, zero mismatch; sec. 4.6 hypothesis exonerated at code caliber per the sorry audit.
- [20] TOE-SYLVA Formalization Governance Group. D1 dual-candidate journal-submission peer review (three-role simulated review). Technical report, TOE-SYLVA repository, 2026. `framework/journal_submission_d1_review.md`; conflict inventory C1–C6, 11-item improvement list (P1 gaps of this draft), novelty verdicts, rollback-incident documentation.
- [21] TOE-SYLVA Formalization Governance Group. D2 journal matching matrix. Technical report, TOE-SYLVA repository, 2026. `framework/journal_matching_matrix.md`; data-driven venue plan: JAR main target, LMCS backup, CPP 2028 conference line; JFR on observation list.
- [22] TOE-SYLVA Formalization Governance Group. Deleted-fabricated-claims ledger and the ai-failure-to-verifiable-protocol. Technical report, TOE-SYLVA repository, 2026. `data/deleted_fabricated_claims.json` (machine-readable; 15 fabricated claims removed in the counting-geometry audit) and `papers/AI_FAILURE_TO_VERIFIABLE_PROTOCOL.md`; four rounds of quality engineering documented in `DEVELOPMENT_DIRECTIONS.md`.
- [23] TOE-SYLVA Formalization Governance Group. Independent compile log for `StratifiedChernNumber.lean` (t3 grading evidence). repository artifact `papers/_/_panel_records/stratified_chern_compile_log_20260810.txt`, 2026. Lean 4.29.0, mathlib4 8a178386, exit 0, 6.7 s; `#print axioms` = standard trio for all six theorems.

- [24] TOE-SYLVA Formalization Governance Group. P0 number-caliber reconciliation report (axiom / agda / proof-rate / sweep accounts). Technical report, TOE-SYLVA repository, 2026. `framework/p0_reconciliation_report.md`; the canonical register: $196 = 30+44+31+29+30+32$; registry 448; residual/live 253; five classes $50/90/17/24/13 + 2$ bundling; Agda 7/25/131; reproduction commands in its appendix.
- [25] TOE-SYLVA Formalization Governance Group. Proof status and governance (claim-tier registry, honest proof-rate definition). Technical report, TOE-SYLVA repository, 2026. `framework/proof_status.md v1.2` (Agda counts corrected to measured 7/25/131 per P0).
- [26] TOE-SYLVA Formalization Governance Group. Quarantine manifest for the batch corpus (`sylva_provenlayer2.lean`). Technical report, TOE-SYLVA repository, 2026. `releases/quarantine/quarantine_manifest.json`; origin path, SHA-256, reason list, exclusion scope.
- [27] TOE-SYLVA Formalization Governance Group. Sorry-contamination audit of the batch corpus (full-population). Technical report, TOE-SYLVA repository, 2026. `framework/sorry_contamination_audit.md`; string caliber $36,199/119,859 = 30.2\%$ reversed to zero code-level; single quarantine case; three-level caliber standard in sec. 6.
- [28] Towards a common framework for autoformalization. arXiv:2509.09810. pipeline architecture reference; TODO(P1-5) formal entry.

A Reproduction Protocol

All canonical numbers in this paper are reproducible from the repository root with the commands below (per the P0 reconciliation report’s appendix; values as of the v7.99 working tree). The honesty gate is `python3 scripts/verify_honest_repo.py --ci` (expected: PASS, exit 0).

A.1 Axiom calibers

- **A (registry 448)**: `python3 -c "import json; print(len(json.load(open('framework/axiom_registry.json'))['axioms']))"`
- **B (live declarations 253)**: `rg -c '^axiom\s' --glob '*.lean' --glob '!.lake' sylva_formalization/ | awk -F: '{s+=$NF} END {print s}'`
- **A' (registry residual $\equiv$ 253)**: registry join against live declarations (Python script in the P0 report appendix): entries (file, name) registered **and** still declared axiom.
- **Evolution chain** (445/399/368/315/285/253): per-commit census via `git grep -c -E '^axiom[\t]' <commit>` -- 'sylva_formalization/**/*.lean' over commits bf6590463, e7b326854, 39105a690, e3026fc0e, ae8ab0e07.

A.2 Agda calibers

- Files: `find sylva_formalization -name '*.agda' | wc -l  $\rightarrow$  7`.
- Top-level blocks: `rg -c '^s*postulate\b' --glob '*.agda' sylva_formalization/ | awk -F: '{s+=$NF} END {print s}'  $\rightarrow$  25`.
- Name-level declarations: per-block parsing $\rightarrow$ 131 (Quantum 53 / Spectrum 36 / Complete 15 / CNFCategory 12 / Order 12 / Cauchy 2 / Field 1).
- The string-level count (55) and the planning value (149) are deprecated calibers, retained only in erratum notes.

A.3 Proof-rate calibers

- Authoritative (3.91% = 22,328/571,611, handcrafted; batch 0% reported separately): `scripts/compute_proof_rate.py` (count_sorry_no_comments reference implementation), dashboard `framework/proof_rate_dashboard.md`.
- Stale snapshot (99.79%/100.00%/79.70%): frozen in `framework/DASHBOARD.md` under a STALE banner; denominator 546,356, axiom count 553 (both old-caliber).

B Glossary of Calibers

- String caliber: substring match over raw file content (comments included). Cheap, reproducible, and the source of the 30.2% false alarm. Never authoritative for placeholder counts.
- Code caliber: statement-level pattern with comments stripped (e.g. `^\s*sorry\b`); the reference level for placeholder scans.
- Cl-pattern caliber: the exact regular expressions used by `verify_honest_repo.py --ci`; the binding level for gate behavior.
- Registry caliber (axioms): entries of `axiom_registry.json` (file×name pairs).
- Declaration caliber (axioms): `axiom` command census over `.lean` files (scope variants: corpus only, or whole repository).
- Sweep caliber (redemptions): per-entry closure records in the sweep reports, summed.
- Name-level caliber (Agda): postulate name declarations, as distinct from top-level `postulate` blocks or substring occurrences.